Self-Stabilizing Leader Election in Regular Graphs

Hsueh-Ping Chen*
Department of Electrical Engineering, National Taiwan University
Taipei, Taiwan
r07921034@ntu.edu.tw

Ho-Lin Chen†
Department of Electrical Engineering, National Taiwan University
Taipei, Taiwan
holinchen@ntu.edu.tw

ABSTRACT

Population protocols [3] are used as a distributed model that captures the behavior of passively mobile agents. Leader election is one of the most well-studied problems in this model. In this paper, we focus on the self-stabilizing leader election (SSLE) problem proposed by Angluin *et al.* [5]. Previously, it is known that SSLE can be performed on arbitrary rings and tori with a constant number of states [11], but SSLE on complete graphs requires $\Omega(n)$ states [9].

In this paper, we propose the first SSLE population protocol for arbitrary k-regular graphs, which solves an open question proposed in [5]. There are two different SSLE protocols in this paper. In both protocols, the number of states is independent of the size of the graph. The first protocol is simpler and more intuitive but requires $O((64c)^k \cdot k^{4k+4})$ states, where c is the constant number of states used by the SSLE protocol for rings [11]. The second protocol is more carefully designed to reduce the number of states to $O(k^{12})$. Both of these two constructions can apply to arbitrary graphs if every node knows its own degree.

CCS CONCEPTS

• **Theory of computation → Distributed algorithms**.

KEYWORDS

leader election, self-stabilizing, population protocols, regular graphs, Euler cycle, spanning tree

ACM Reference Format:
Hsueh-Ping Chen and Ho-Lin Chen. 2020. Self-Stabilizing Leader Election in Regular Graphs. In *ACM Symposium on Principles of Distributed Computing (PODC '20), August 3–7, 2020, Virtual Event, Italy*. ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3382734.3405733

1 INTRODUCTION

Population protocols [3] are used as a distributed model that captures the behavior of passively mobile agents. In the model, agents have local states and can communicate with each other via pair-wise interactions. Like conventional distributed models, there is an underlying communication graph. Every agent can only communicate with its neighbors in the communication graph.

The computational power of population protocols is much weaker than conventional distributed models due to the following settings. First, agents are passively mobile in the sense that they cannot actively choose to interact with a particular neighbor in the graph. There exists an unpredictable scheduler which governs the interactions among the entire population. Second, the protocol is anonymous, which means that agents cannot distinguish one agent from another by a global ID. Moreover, the model does not have a local orientation, so agents cannot differentiate among their neighbors. Although these assumptions make population protocols a much weaker model, it can still achieve some interesting tasks when the scheduler obeys certain fairness conditions. Example applications include calculating the average temperature of an animal population and modeling the chemical reactions of molecules. On the other hand, the similarity between population protocols and other computation models, such as Petri Nets [19], vector addition systems [18], and chemical reaction networks [10], make it receive more attention.

In this paper, we focus on solving an important distributed computing task, called leader election, in population protocols. Leader Election has long been an important research problem in general distributed models since it can serve as a basic building block for many distributed algorithms. When a leader is elected, it can coordinate the communication between different computing units, and make it much easier to design algorithms. As for population protocols, leader election is also a dominant research topic. Many works [1, 2, 7, 13, 15, 16], has focused on how to solve leader election efficiently. It is also known [4] that many problems can be solved quickly with the help of a leader.

In this work, we focus on the self-stabilizing leader election (SSLE) problem. Self-stabilization was proposed in [12]. A distributed algorithm is self-stabilizing if the correct final output will be reached regardless of the initial configuration. Therefore, a self-stabilizing algorithm can tolerate any finite number of transient failures, making it immune to unexpected errors that can happen in practical distributed systems. For population protocols, self-stabilization leader election (SSLE) is first discussed in [5]. Since then, many works [6, 8, 11, 13, 14] devoted to solving SSLE under different scenarios. Among these works, the topic mostly related to this work is SSLE on rings. In [5], an SSLE protocol on rings of arbitrary odd size was proposed, and solving SSLE on rings of arbitrary size was listed as an open question. In [14], the authors presented a protocol that can achieve SSLE on all rings given the presence of a special oracle that can detect the presence of a leader

*Research supported by MOST (Taiwan) grant number 107-2221-E-002-031-MY3.
†Corresponding author. Research supported by MOST (Taiwan) grant number 107-2221-E-002-031-MY3.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '20, August 3–7, 2020, Virtual Event, Italy
© 2020 Association for Computing Machinery.
ACM ISBN 978-1-4503-7582-5/20/08…$15.00
https://doi.org/10.1145/3382734.3405733

in the ring. Recently, Chen *et al.* [11] proposed a protocol that solves SSLE on all rings using a constant number of states. The constant state complexity is achieved by implementing a leader detector with the Thue-Morse Sequence. The protocol works for all fair executions, but may require $O(2^{n^2 log(n)})$ time even if updates happen uniformly at random. This protocol is used as a subroutine in our constructions. We refer to their protocol as the Ring-Protocol in later paragraphs. This paper focuses on the problem of SSLE on general k-regular graphs. This is an open problem proposed in [5]. Prior to our work, there were no known results for this problem on population protocols. Itkis *et al.* [17] designed an SSLE algorithm for general k-regular graphs in a different distributed computing protocol. The protocol first constructs a spanning tree and then defines the root as the leader. In the original model of [17], each node can observe the states of all its neighbors at the same time, and the model has a local orientation. Hence, there is no trivial way to simulate the original model by population protocols; in fact, the original model can do SSLE on general graphs, which is impossible for population protocols.

1.1 Our contribution

In this paper, we propose the first SSLE population protocol for arbitrary k-regular graphs, which solves an open question proposed in [5]. There are two different constructions in this paper. The first protocol doubles the number of edges, decomposes each node into several virtual nodes and decomposes the graph into a collection of rings. The protocol designed in [11] is used to elect a leader in each ring. In the case that there is only one ring, the protocol stabilizes to a configuration with a single leader. Otherwise, the system modifies the configuration arbitrarily. The number of states used in this protocol is independent of the size of the graph but is exponential in the degree k of the regular graph ($O((64c)^k \cdot k^{4k+4})$), where c is the constant number of states of the Ring-Protocol. Although this protocol is quite simple and intuitive, it may not be a good protocol for regular graphs with a high degree. The second protocol is more complicated to describe, but has a much better state complexity. The protocol uses the idea of the SSLE algorithm in [17] and creates a spanning tree with the root being the leader. For k-regular graphs, this protocol can perform SSLE with $O(k^{12})$ states. This state complexity is much closer to the $\Omega(k)$-states lower bound implied by the results for complete graphs [9]. Both of these two constructions can apply to arbitrary graphs if every node knows its own degree.

In the design of the two protocols of this paper, the agents check the current configuration to see if it is the desired ring/tree structure. When an error is identified, instead of directly fixing the error, the agents just randomly switch to some state and repeatedly check the new configuration. This approach greatly reduces the intermediate states required and makes the design and verification of the protocol much easier, especially when we do not need to worry about some complicated combination of these intermediate states showing up in the initial configuration. The main drawback of this type of approach is that the stabilizing time could be very long. In this work, we only assume global fairness on the execution sequence and thus the stabilizing time is not the main concern.

2 PRELIMINARIES

2.1 Population Protocols

In this section, we recap the definition of population protocols proposed in [3]. A population protocol is a distributed computing model defined on a communication graph, where each node can only communicate with its neighbors. Population protocols are anonymous, which means each node in the communication graph does not have a unique ID. Furthermore, the model has no local orientation, so a node cannot differentiate among its neighbors. The latter assumption makes the design of population protocols more difficult than conventional distributed models.

A population protocol P defined on a graph $G = (V, E)$ can be represented by a 6-tuple (X, i, δ, Y, o, Q). Initially, each node is given an input $x \in X$, where X is the input alphabet. Q is the set of possible states for each node, and $i : X \to Q$ is the input function, which maps an input of a node to its initial state. In each step, two adjacent nodes are picked by a scheduler, one as the initiator of the interaction, and the other as the responder. The two nodes update their states by the non-deterministic transition function $\delta : Q \times Q \to P(Q \times Q) - \phi$, where $P(Q \times Q)$ is the power set of $Q \times Q$, and ϕ means an empty set. For example, consider an interaction where the states of the initiator and the responder are p, q. After the interaction, their states can become r, s if and only if $(r, s) \in \delta(p, q)$. People can always observe the output of each node by doing the output mapping $o : Q \to Y$, where Y is the output alphabet.

A state configuration of a population protocol is the global state of the communication graph, which can be easily described by specifying the state of each node in the graph. Equivalently, one can view a state configuration as a mapping $C : V \to Q$, which maps each node to its corresponding state. There is a transition between two state configurations C_1 and C_2, denoted by $C_1 \to C_2$ if and only if there are two nodes $\{u, v\} \in E$ such that $(C_2(u), C_2(v)) \in \delta(C_1(u), C_1(v))$ and $C_2(x) = C_1(x)$ for $x \in V - \{u, v\}$. A state configuration C' is reachable from another state configuration C if there exist finite number k of state configurations $C_1, C_2, ..., C_k$ such that $C_i \to C_{i+1}$ for $i = 1, 2, ..., k-1$ and $C = C_1, C' = C_k$.

A problem defined on a communication graph has two components (ICs, F), ICs is the set of possible input configurations, and F is a function that maps each input configuration to a set of valid output configurations. An input configuration is a mapping $I : V \to X$ that describes the input of each node, and an output configuration is a mapping $O : V \to Y$ that maps each node to its corresponding output. An input configuration can be easily transformed into its corresponding state configuration, and a state configuration can also be transformed into its output configuration. Abuse of notation, we use $i(S)$ to denote a state configuration corresponding to the input configuration S, and $o(C)$ to represent an output configuration corresponding to the state configuration C.

An execution of a population protocol given input configuration S, is an infinite sequence of configurations $C_0, C_1, C_2, ...$ where $i(S) = C_0$ and $C_j \to C_{j+1}$ for $j = 1, 2, ...$ For most cases, a population protocol has no stopping configurations, and an execution of a protocol would continue forever. It means that one cannot wait for the protocol to stop and watch its output. Instead, the stability of a population protocol is defined. A state configuration

C is output-stable if for every configuration C' reachable from C, $o(C) = o(C')$. A protocol solves a problem (ICs, F), if for every input configuration $S \in ICs$, the execution of the protocol given input configuration S will eventually reach an output-stable configuration C and $o(C) \in F(S)$.

A scheduler determines which two adjacent nodes will participate in the next interaction. Usually, a scheduler has unpredictable behavior, but to be realistic, the scheduler must obey some fairness condition; otherwise, it is impossible to do any task under a complete adversary scheduler which only keeps two particular nodes interact with each other forever. In this work, all discussion is under global fairness, which states that if a configuration is always reachable in an execution, it will be reached eventually. This fairness condition can make sure that a scheduler cannot skip some configurations forever if they are always reachable.

2.2 Self-stabilizing Leader Election

For a self-stabilizing leader election (SSLE) problem, the input alphabet, X, is exactly the state set Q, and the input function i is an identity function. All the possible input configurations are the entire set of state configurations, that is, the starting configuration of an execution of the protocol can be an arbitrary configuration. On the other hand, the output alphabet is $\{L, N\}$, where L stands for a leader, and N stands for a non-leader. A configuration O is a valid output configuration if there is exactly one node $v \in V$ such that $O(v) = L$ and for any $u \neq v$, $O(u) = N$, which means there is only one leader in a valid output configuration.

2.3 Detect Error and Random Change

Here, we present a technique for designing a self-stabilizing protocol, called detect-error-and-random-change. The idea is straightforward. Invalid configurations are classified into two types: 1) Erroneous configurations and 2) Error-free configurations. For error-free configurations, the protocol would use detailed steps to move them to a valid output-stable configuration; but for erroneous configurations, the protocol would change them to an error-detected configuration first. An error-detected configuration could reach any possible configuration in the protocol by randomly changing the agents' states, which implies that it could always reach a valid output-stable configuration. Given that the scheduler obeys global fairness, an invalid configuration can reach a valid output-stable configuration eventually. The detailed description is given below.

When an erroneous configuration is detected, an error signal (a boolean state variable $errFlag = True$) would be broadcast to the entire graph. Each node with $errFlag = True$, called an error-detected node, would randomly change its state and randomly decide to reset the $errFlag$ to be $False$. Any configuration with an error-detected node is called an error-detected configuration; one can easily verify the result that an error-detected configuration could reach all possible configurations of the protocol.

Lemma 2.1 (Detect-Error-and-Random-Change-Lemma). *For a population protocol, if every invalid configuration meets the requirement of either of the following two conditions, then it is a self-stabilizing solution to the corresponding problem. 1) It is erroneous, and it could reach an error-detected configuration. 2) It is error-free, and it could reach a valid output-stable configuration.*

3 SSLE ON REGULAR GRAPHS

In this section, we present a population protocol for SSLE on regular graphs. In this construction, the protocol decomposes each node into several virtual nodes and decomposes the graph into a collection of rings. The protocol designed in [11] is used to elect a leader in each ring. In the case that there is only one ring (corresponding to an Euler cycle), the protocol stabilizes to a configuration with a single leader. Otherwise, multiple leaders will be generated. When these leaders detect each other, they send out error signals which allow the system to modify the configuration arbitrarily. In section 3.1, we first describe a protocol that works for even-degree regular graphs, in which an Euler cycle exists. In section 3.5, we extend our construction to odd-degree regular graphs. For general graphs, given the assumption of each node knowing its degree, our result could also apply.

3.1 Euler Cycle Construction in Even-Degree Regular Graphs

Consider building a virtual ring on a $2k$-regular graph. The first step is dividing each node into k virtual nodes. Each virtual node randomly picks two virtual nodes from neighboring nodes as its virtual neighbors in the virtual ring: one as its predecessor, and the other as its successor. Although each virtual node selects their virtual neighbors randomly, the protocol will verify whether their choices are coherent, that is, if virtual node A picks virtual node B as its successor, then B should pick A as its predecessor. Since each virtual node need to identify its virtual neighbors in the virtual ring, local IDs are required for a node to differentiate among its neighbors. Here, 2-hop coloring [20] is used to do local orientation The details will be explained in section 3.2.

When the selection of all virtual nodes is coherent, the constructed virtual graphs have the following two possibilities. 1) Single virtual ring: The Ring-Protocol running on this ring may select exactly one virtual node as the leader, and the node containing this virtual node is the unique leader in the original graph. If this process is completed before the ring decomposition gets modified, the leader election is completed. 2) Multiple virtual rings: Each virtual ring would generate a leader, causing multiple leaders in the graph. This invalid configuration can be detected by making each leader do token circulation on its virtual ring. When two tokens travel to adjacent nodes, the protocol can detect it and set an error flag which randomly changes the ring decomposition.

SS-EulerCycle is the formal protocol of the above description. The state variables are defined below. *color* is the local ID of each node, and *(prevColor,colorBit)* are used in the sub-protocol SS-TwoHop. Each node contains k virtual nodes, $\{node[i]\}_{i=1}^{k}$, where each virtual node $node[i]$ has its state variables: *(ringState, predcessorColor, successorColor, token, bit). ringState* is the state of the Ring-Protocol. A node is output as a leader if it contains a virtual node whose *ringState* is a leader in the Ring-Protocol. *predecessorColor* and *successorColor* record the virtual neighbors, and $token$ and bit are used for token circulation.

$ErrFlag$ is a boolean variable for detect-error-and-random-change (see preliminaries for more details). Once an error is detected, the errFlag is set and the error signal can be broadcast to the entire graph. When receiving this signal, each node will randomly change

its state and randomly do $errFlag \leftarrow False$ to stop the random-state-change process. The ring decomposition will be modified arbitrarily in this process. There is always a chance that all nodes change their states into the correct states in the valid output-stable configuration. (The existence of a valid output-stable configuration will be proved later.) Since the valid output-stable configuration is always reachable, the global-fairness property of the scheduler guarantees the valid configuration would be reached eventually.

Protocol 1 SS-EulerCycle

state variables: $(\{node[i]\}_{i=1}^{k}, color, prevColor, colorBit, errFlag)$

$node[i] : (ringState, predcessorColor, successorColor, token, bit)$

$color, prevColor, predcessorColor, successorColor \in \{1, ..., 4k^2\}$

$colorBit,\ \ errFlag,\ \ token,\ \ bit \in \{True, False\}$

Input: node u is the initiator, node v is the responder

SS-TwoHop(u, v)
Find i, j such that $(u.node[i].successorColor = v.color) \wedge$
$(v.node[j].predecessorColor = u.color)$
if such i, j exists **then**
{ δ_{ring} below is the transition function of the Ring-Protocol}
$(u.node[i].ringState, v.node[j].ringState) \leftarrow$
$\delta_{ring}(u.node[i].ringState, v.node[j].ringState)$
SS-Token$(u.node[i], v.node[j])$
else
$u.errFlag \leftarrow True$ {Make sure coherence in the virtual ring.}
if There exist p, q such that
$v.node[p].token = u.node[q].token = True$ **then**
$u.errFlag \leftarrow True$ {Detect multiple leaders.}
if $u.errFlag = True$ **then**
$u \leftarrow RandState$
{RandState is a randomly-chosen state from the state set.}
$(u.errFlag, v.errFlag) \leftarrow (RandBit1, RandBit2)$
{RandBit1 and RandBit2 are independent random bits.}
{Detect-Error-and-Random-Change}

3.2 Two-hop Coloring

Since population protocols have no local orientation, we would firstly assign local IDs to nodes for the ease of protocol design. For this task, we use a 2-hop coloring protocol introduced in [20]. The sub-protocol, SS-TwoHop, exactly follows the previous protocol except for some changes of variable names to fit other constructions in this paper. As its name suggests, 2-hop coloring assigns a color to each node and every two nodes with distance exactly two will have different colors. It can be solved in the following self-stabilizing protocol. Each node records its color (state variable: color) and the color of the node it interacted with in the previous interaction (state variable: prevColor). When two nodes interact, the protocol would generate a single random bit and assign it to the boolean variable, colorBit, of each node. Therefore, if two nodes, A and B, have matching colors (A's prevColor = B's color and B's prevColor = A's color), their colorBits should be the same. Otherwise, it means that the protocol has not yet stabilized, and the colors of both nodes would be randomly changed. We would use two examples to explain how the protocol detects color conflict in an invalid configuration and how the colors stop changing when the coloring is valid.

1) Invalid configurations: If a node, A, has two neighbors, B and C, with the same color, the protocol could detect this color conflict by the following interaction order: (A, B), (A, C), (A, B). After the first two interactions, node A has the matching color with both B and C since B and C have the same color. However, the colorBit of B may not equal to the colorBit of A, and the third interaction could randomly change the colors of A and B when the protocol finds two agents with matching colors but different colorBits.

2) Valid configurations: Consider the case that every two nodes with distance two have different colors. The only possibility for two nodes having matching colors is that two nodes interact consecutively. In this case, these two nodes must have identical colorBits, so their colors would never change.

Protocol 2 SS-TwoHop

Input: node u is the initiator, node v is the responder
if $(u.color, v.prevColor, u.colorBit) =$
$(v.prevColor, v.color, \neg v.colorBit)$ **then**
$(v.color, u.color) \leftarrow (RandColor1, RandColor2)$
{RandColor1, RandColor2 are random colors}
$(u.prevColor, v.prevColor, u.colorBit, v.colorBit) \leftarrow$
$(v.color, u.color, RandBit, RandBit)$
{RandBit is a random bit.}

3.3 Token Circulation

In [5], the authors propose a protocol to do self-stabilizing token circulation in an oriented ring given there exists a unique leader in the starting configuration. The sub-protocol, SS-Token, exactly follows the previous protocol except for some changes of variable names. The idea is straightforward. The leader would transfer its bit along one direction of the ring. When the bit goes back to itself, the leader flips the bit and transfer it again. By setting a token at the node whose predecessor has complemented bit value, a token would circulate along the ring as the bit of the leader is repeatedly transferred.

Protocol 3 SS-Token

Input: node u is the initiator, node v is the responder
Since ringState is the state of an SSLE protocol, it could be output as a leader (L) or non-leader (N).
if $o(v.ringState) = N \wedge v.bit \neq u.bit$ **then**
$v.token \leftarrow True,\ \ u.token \leftarrow False,\ \ v.bit \leftarrow u.bit$
if $o(v.ringState) = L \wedge u.bit = v.bit$ **then**
$v.token \leftarrow True,\ \ u.token \leftarrow False,\ \ v.bit \leftarrow \neg v.bit$

3.4 Proof of Correctness and State Complexity Analysis

By the Detect-Error-and-Random-Change-Lemma, only two conditions need to be checked for proving the correctness of SS-EulerCycle. The two conditions are verified by the following two lemmas.

Lemma 3.1. *There exists a valid output-stable configuration of SS-EulerCycle.*

Proof. We would explain what the valid output-stable configuration looks like and verify it is output-stable at the same time. In the valid output-stable configuration, colors of all nodes satisfy two-hop coloring and would never change, all the virtual nodes form a single virtual ring, and there exists a unique token in the ring. The existence of a stable colored configuration follows directly from the correctness of the sub-protocol SS-TwoHop, and a stable configuration for token-circulation follows from the correctness of the sub-protocol SS-Token. Since the correctness of the two sub-protocols is verified in [5, 20], what remains to be proved is a stable leader in the virtual ring.

Since an even-degree regular graph always has an Euler cycle, this virtual ring does exist. For the *RingStates* of all virtual nodes on this ring, this ring-configuration is a valid output-stable configuration for the Ring-Protocol, meaning that there is exactly one virtual node whose output is a leader of the Ring-Protocol, and no leader will be created or removed by the Ring-Protocol for all configurations reachable. The existence of such stable ring-configuration is verified in [11]. The output function of SS-EulerCycle would define the node containing this virtual leader as the leader, and this configuration is valid since there is exactly one leader in the communication graph. □

Lemma 3.2. *For each invalid configuration of SS-EulerCycle, it is either one of the following two cases. 1) It is error-free, and it can reach a valid output-stable configuration. 2) It is erroneous, and it can reach an error-detected configuration C where $C(v).errFlag = True$ for some $v \in V$.*

Proof. If two nodes break the coherence of virtual rings, the configuration is erroneous. It could be easily detected when the two nodes interact. Therefore, an error-detected configuration is reachable. Hence, in the following, we can discuss under the condition that all nodes obey the coherence of virtual rings, that is, if node A chooses node B as its virtual predecessor, then node B should pick node A as its virtual successor.

Since all nodes obey the coherence of virtual rings, the resulting graphs may be a single virtual ring or multiple virtual rings. In the first case, the configuration is error-free. The Ring-Protocol would elect a unique virtual leader in the ring, and the node containing this virtual leader will be the unique leader of the original graph. Therefore, a valid output-stable configuration is reachable. For the second case, the configuration is erroneous. The Ring-Protocol will produce one virtual leader for one virtual ring. Hence, there will be multiple virtual leaders in the graph and they will all execute token circulation on their rings. Since the graph is connected, a configuration where two tokens are located at the adjacent nodes is reachable. In this case, the errFlag of one node will be set, and the error-detected configuration is reached. □

Theorem 3.3. *SS-EulerCycle is a self-stabilizing population protocol for leader election on even-degree regular graphs.*

Proof. Given lemma 3.1 and lemma 3.2, this theorem can be implied by the Detect-Error-and-Random-Change-Lemma. □

After proving that SS-EulerCycle can do SSLE on even-degree regular graphs, we can do a simple calculation to analyze the number of states used by this protocol. For a 2k-regular graph, $4k^2$ colors are enough for 2-hop coloring. Each virtual node has the state variables *(ringState, predcessorColor, successorColor, token, bit)*. Let c denote the number of different values of *ringState*. *token* and *bit* are both boolean variables. Hence, the number of different values of a virtual node is $c \cdot 4k^2 \cdot 4k^2 \cdot 2 \cdot 2 = 64ck^4$. There are k virtual nodes for each node, so there are $O((64c)^k \cdot k^{4k})$ different values for the virtual nodes state variables. The remaining state component can be bounded by $O(k^4)$. In conclusion, the state complexity of SS-EulerCycle is $O((64c)^k \cdot k^{4k+4})$.

3.5 Extension

Although there is no Euler cycle in odd-degree regular graphs, we can do a little trick to solve this problem. For a (2k+1)-deg regular graph, we can view each edge as double edges, and divide each node into (2k+1) virtual nodes. In this way, the degree of each node remains even, and we can still build a virtual ring and do SSLE by the same technique introduced in this section.

Furthermore, this idea can also be extended to general graphs. From [5], it is well known that it is impossible to do SSLE in general graphs. However, consider the case where each node knows its own degree; this degree is kept as a state field at the starting configuration, and it would never be changed during executions of the protocol. By doubling edges, each node has even degree, and a virtual ring (an Euler cycle) could be built as what we do in this section. One more thing worth noting is that nodes with different degrees would have different state sets since they have different numbers of virtual nodes. Therefore, when using detect-error-and-random-change, the state of each node should only randomly change to the state in the corresponding state set.

4 SSLE PROTOCOL WITH $O(k^{12})$ STATES

In section 3, the number of states can be as large as $O((64c)^k \cdot k^{4k+4})$. Although it is independent of the size of the graph, it is still bad for regular graphs with large degrees, such as complete graphs. In this section, we present a different protocol. Using the idea in [17], each node either randomly picks one of its neighbors as its parent or becomes the root. The graph is decomposed into a collection of rooted trees. The system may stabilize when the graph contains exactly one single-rooted tree and will reach an error state and reset otherwise. In the model of [17], a node can update its state by observing the states of all of its neighbors. But in population protocols, nodes can only do pair-wise interaction. To simulate the original model using population protocols, one may be attempted to make each node record states of its neighbors one by one, and then use this information to update its state. However, to record the state of each neighbor, the state complexity would also be $\Omega(exp(k))$, which is not an improvement. Hence, our main contribution here is a special technique that can implement the main idea of the original protocol with only $O(k^{12})$ states.

4.1 Self-stabilizing Spanning Tree

4.1.1 Intuitive Idea. SS-Tree is a self-stabilizing protocol to build a spanning tree of the communication graph, and the root of the

tree is the leader of the population. In the spanning tree, each node should be able to identify its parent, so assigning local IDs to nodes is necessary. Again, we use two-hop coloring as in section 3.

Every node randomly picks one of its neighbors as its parent by recording the corresponding color, or it can claim itself as the root. By viewing each parent-child pair as a directed edge, one can find a corresponding pedigree graph T. Then we use two sub-protocols to help to transform T to a spanning tree of the original graph. First, CycleDetector can detect cycles in T, and set the errFlag. Second, MultipleRootDetector will find out whether there are more than one root in T. If multiple roots are detected, the protocol will reflect this by setting the errFlag. When both cycles and multiple roots are ruled out, the only remaining possibility of T is a single spanning tree of the original graph. The details of both sub-protocols will be discussed later, and now we will focus on the technique that can dramatically improve the state complexity.

4.1.2 main technique. Since the protocol is required to be self-stabilizing, it is possible that at the starting configuration, some node record a ghost parent, which means it records a parentColor which is not equal to the color of any of its neighbors. Making each node record all the colors of its neighbors may solve this problem, but it will take $\Omega(k^k)$ states (where k is the degree of the regular graph). Therefore, a different approach is used here. First, each color is represented by a unique positive integer, which naturally defines a total ordering on all colors. When a node wants to ensure that its parentColor is the color of one of its neighbors, it would check the color of its neighbors in the order of increasing color. Also, the node would keep a counter to count how many neighbors it has already checked. If the counter of a node counts to the degree of the graph without finding any node with parentColor, a ghost parent can be detected. By doing so, a node can detect a ghost parent without memorizing all the colors of its neighbors at the same time. Using this technique, one can check some basic predicate of states of all the neighbors of a node using only $poly(k)$ states. In the sub-protocol CycleDetector, this technique will be used again.

4.2 Cycle Detector

CycleDetector utilizes the Ring-Protocol [11] to detect a cycle. In the following paragraph, we would give an intuitive idea for detecting a cycle.

At any time, the protocol would randomly mark a path in the pedigree graph T using the following way. First, the protocol randomly marks a starting node by setting its state variable *loopMarker* to be *True*. The starting node could be marked only if each of its neighbors has not been marked. This condition can be checked by using the main technique introduced in section 4.1 and the state variables *loopCounter*, *loopColor* are used for this purpose. Afterwards, the protocol could extend the path by marking the parent of the ending node of the current path. The boolean state variable *loopChild* represents whether a node has a child in the path, and the boolean state variable *loopParent* represents whether a node has a parent in the path. Therefore, a marked node is the starting node of a path if its state variable $loopChild = False$; a marked node is the ending node of a path if its state variable $loopParent = False$.

Protocol 4 SS-Tree

state variables:
$color$, $parentColor$, $prevColor \in \{1, 2, ..., k^2\}$
$counterColor \in \{0, 1, ..., k^2\}$
$counter \in \{0, 1, ..., k\}$; $errFlag$, $root \in \{True, False\}$
Input: node u is the initiator, node v is the responder
SS-TwoHop(u, v)
Randomly: $u.counter \leftarrow 0, u.counterColor \leftarrow 0$
if $u.counterColor < v.color \wedge u.root = False$ **then**
$u.counterColor \leftarrow v.color, u.counter \leftarrow u.counter + 1$
if $v.color = u.parentColor$ **then**
$u.counter \leftarrow 0$
if $u.counter = k$ **then**
$errFlag \leftarrow True$ {Detect ghost parent.}
CycleDetector(u, v)
MultipleRootDetector(u, v)
if $u.errFlag = True$ **then**
$u \leftarrow RandomState$
$(u.errFlag, v.errFlag) \leftarrow (RandomBit1, RandomBit2)$
{Detect-Error-and-Random-Change}

When each node is marked, the state variable *ringState* is copied to the variable *ringStateCopy*. Then the protocol runs the Ring-Protocol on any two nodes adjacent on the same marked path, and all the state transitions caused by the Ring-Protocol affects only the variable *ringStateCopy*. When a cycle in T is marked by the protocol, the Ring-Protocol could generate a ring-leader (for *ringStateCopy*) on this cycle. A ring-leader being generated means the detection of a cycle, and the errFlag will be set.

4.3 Multiple-root Detector

The sub-protocol, MultipleRootDetector, detects multiple roots in the following way. Each root would toss a coin, and send the coin by a random walk on the pedigree tree corresponding to this root. When a random walk returns to the root, the root would toss a coin again. By repeatedly doing so, if there are more than one roots in the pedigree graph, there must be a time when two roots toss different coin values and the two coins travel to adjacent nodes. After the protocol finds two nodes with distinct coin values, multiple roots are detected and the protocol will set the errFlag.

4.4 Proof of Correctness and State Complexity Analysis

Verifying the correctness of SS-Tree is similar to what we have done for SS-EulerCycle in section 3. The two conditions of Detect-Error-and-Random-Change-Lemma would be checked by the following two lemmas.

Lemma 4.1. *There exists a valid output-stable configuration for SS-Tree.*

Proof. As in the proof of lemma 3.1, we would describe the valid output-stable configuration and verify it. In the valid output-stable configuration of SS-Tree, the pedigree graph, which determined by the state variable color and parentColor, is a spanning tree of the communication graph, which rule out the possibility of a

Protocol 5 CycleDetector

state variables:
$ringState$, $ringStateCopy$
$loopMarker$, $loopChild$, $loopParent \in \{True, False\}$
$loopColor \in \{0, 1, ..., k^2\}$, $loopCounter \in \{0, 1, ..., k\}$
Input: node u is the initiator, node v is the responder
Randomly Do: $u.loopCounter \leftarrow 0$, $u.loopColor \leftarrow 0$
if $u.loopColor < v.color$ **then**
$u.loopColor \leftarrow v.color, u.loopCounter \leftarrow u.loopCounter + 1$
if $v.loopMarker = True$ **then**
$u.loopCounter \leftarrow 0$
if $u.loopCounter = k$ **then**
$u.loopMarker \leftarrow True$, $u.loopChild \leftarrow False$
$u.ringStateCopy \leftarrow u.ringState$
$u.loopPatent \leftarrow False$
{Mark a starting node of a path.}
Randomly Do: $u.loopMarker \leftarrow False$
if $u.color = v.parentColor$ **then**
if $(u.loopMarker = v.loopMarker = True) \wedge$
$(u.loopChild = v.loopParent = True)$ **then**
$(u.ringStateCopy, v.ringStateCopy) \leftarrow$
$\delta_{ring}(u.ringStateCopy, v.ringStateCopy)$
if $(u.loopMarker = False) \wedge (v.loopMarker = True) \wedge$
$(v.loopParent = False)$ **then**
$u.loopChild \leftarrow True$, $v.loopParent \leftarrow True$
$u.ringStateCopy \leftarrow u.ringState$
$u.loopMarker \leftarrow True$
{Extend a path.}
if $(u.loopMarker = v.loopMarker = True) \wedge$
$(u.loopChild = False) \wedge (v.loopParent = False)$ **then**
$u.loopChild \leftarrow True$, $v.loopParent \leftarrow True$
{Two paths form a single path or a path forms a cycle.}
if $o(u.ringStateCopy) = leader$ **then**
$u.errFlag \leftarrow True$

Protocol 6 MultipleRootDetector

state variables:
$root \in \{True, False\}$, $coin \in \{-2, -1, 0, 1, 2\}$
Input: node u is the initiator, node v is the responder
if $u.color = v.parentColor$ **then**
if $u.root = True \wedge u.coin = 0$ **then**
$u.coin \leftarrow random\{+1, -1\}$ {toss the coin randomly}
if $|u.coin| = 1 \wedge v.coin = 0$ **then**
$v.coin \leftarrow u.coin$, $u.coin \leftarrow 2 \cdot u.coin$
{the coin randomly walks forward}
if $|u.coin| = 2 \wedge u.coin = 2 \cdot v.coin$ **then**
Randomly Do: $u.coin \leftarrow v.coin$, $v.coin \leftarrow 0$
{the coin walks back}
if $u.coin \cdot v.coin < 0$ **then**
$errFlag \leftarrow True$

ghost parent. Because there is only one root in the pedigree graph, multipleRootDetector will not detect an error. The most important part of this proof is to explain what the ringState of each node should be, and why CycleDetector would not set the errFlag under this setting.

First, we want to show that in the Ring-Protocol, for any given positive integer n, one can find a chain of n nodes with specified ringStates such that the Ring-Protocol can never generate a leader from this chain-configuration. How to obtain this chain is easy. One can simply run the Ring-Protocol on a ring of (n+1)-nodes with arbitrary starting configuration. When a valid output-stable configuration of the Ring-Protocol is reached, one can remove the leader node and get the chain that could never generate a leader.

Now, for the valid output-stable configuration of SS-Tree, one can define a path for each leaf of the pedigree graph by tracing back from the leaf to the root. For every such path, the ringStates of the nodes of the path form a chain-configuration that could never generate a leader. Since the CycleDetector would pick a path in the spanning tree and try if it can generate a leader by executing the Ring-Protocol, with the above design, there is no way for the CycleDetector to generate a ring leader and set the errFlag in the valid output-stable configuration.

Given that the ghost parent is ruled out and the two sub-protocols would not detect errors, the protocol would keep the unique leader forever. Hence, the valid output-stable configuration does exist. □

Lemma 4.2. *For each invalid configuration of SS-Tree, it is either of the following two cases. 1) It is error-free, and it can reach a valid output-stable configuration. 2) It is erroneous, and it can reach an error-detected configuration C where $C(v).errFlag = True$ for some $v \in V$.*

Proof. There are two kinds of invalid configuration: no leader and multiple leaders. Either case, the configuration is erroneous. If there is no leader in the configuration, which means there is no root in the pedigree graph, the pedigree graph must contain a cycle. This cycle can be detected by the sub-protocol CycleDetector, and an error-detected configuration is reached. For the case where there are multiple leaders in the configuration, the sub-protocol, MultipleRootDetector, could detect it. Therefore, for both cases, an error-detected configuration is reachable. □

Theorem 4.3. *SS-Tree is a self-stabilizing population protocol for leader election on regular graphs.*

Proof. Given lemma 4.1 and lemma 4.2, this theorem can be implied by the Detect-Error-and-Random-Change-Lemma. □

The calculation of the state complexity is simple. For each node, it needs to store the following state variables related to the degree of the regular graph: color, parentColor, prevColor, counterColor, loopColor, counter, loopCounter. Each color has $O(k^2)$ different values, and each counter needs $O(k)$ different values. Since the protocol needs five color-variables and two counter-variables, the state complexity of SS-Tree is $O(k^{12})$. This is a huge improvement compared to SS-EulerCycle.

As in section 3, SS-Tree could be extended to general graphs when each node knows its degree. In this case, the state complexity of SS-Tree would become $O(m^{12})$, where m is the maximum degree of the given graph.

5 CONCLUSION

In this paper, we propose two different protocols, SS-EulerCycle and SS-Tree, to solve SSLE on regular graphs. SS-EulerCycle builds a virtual ring in the given regular graph and applies the Ring-Protocol to do leader election. The merits of SS-EulerCycle are its clear description and simple intuitive idea. Besides, how it utilizes the Ring-Protocol [11] shows a deep connection between solving SSLE on regular graphs and on rings. However, the state complexity depends exponentially on the degree of the graph, which may be too expensive for practical use. That is why we design a more complicated protocol, SS-Tree, which has a much better state complexity that depends only polynomially on the degree of the graph. In the second protocol, the idea of electing a leader by constructing a spanning tree follows from [17], but implementing it using population protocols requires a special technique to reduce the state complexity.

For future works, closing the gap between the current $O(k^{12})$ state complexity and the $\Omega(k)$ lower bound is an interesting problem. Also, although we ignore the time complexity of the protocol and only focus on fair execution sequences in this paper, a time-efficient protocol to solve SSLE under random updates might be very useful in more practical situations.

REFERENCES

[1] Dan Alistarh, James Aspnes, David Eisenstat, Rati Gelashvili, and Ronald L Rivest. 2017. Time-space trade-offs in population protocols. In *Proceedings of the twenty-eighth annual ACM-SIAM symposium on discrete algorithms*. SIAM, 2560–2579.

[2] Dan Alistarh and Rati Gelashvili. 2015. Polylogarithmic-time leader election in population protocols. In *International Colloquium on Automata, Languages, and Programming*. Springer, 479–491.

[3] Dana Angluin, James Aspnes, Zoë Diamadi, Michael J Fischer, and René Peralta. 2006. Computation in networks of passively mobile finite-state sensors. *Distributed computing* 18, 4 (2006), 235–253.

[4] Dana Angluin, James Aspnes, and David Eisenstat. 2008. Fast computation by population protocols with a leader. *Distributed Computing* 21, 3 (2008), 183–199.

[5] Dana Angluin, James Aspnes, Michael J Fischer, and Hong Jiang. 2008. Self-stabilizing population protocols. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)* 3, 4 (2008), 13.

[6] Joffroy Beauquier, Peva Blanchard, and Janna Burman. 2013. Self-stabilizing leader election in population protocols over arbitrary communication graphs. In *International Conference On Principles Of Distributed Systems*. Springer, 38–52.

[7] Petra Berenbrink, Dominik Kaaser, Peter Kling, and Lena Otterbach. 2018. Simple and efficient leader election. In *1st Symposium on Simplicity in Algorithms (SOSA 2018)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.

[8] Janna Burman, David Doty, Thomas Nowak, Eric E Severson, and Chuan Xu. 2019. Efficient self-stabilizing leader election in population protocols. *arXiv preprint arXiv:1907.06068* (2019).

[9] Shukai Cai, Taisuke Izumi, and Koichi Wada. 2012. How to prove impossibility under global fairness: On space complexity of self-stabilizing leader election on a population protocol model. *Theory of Computing Systems* 50, 3 (2012), 433–445.

[10] Ho-Lin Chen, David Doty, and David Soloveichik. 2014. Deterministic function computation with chemical reaction networks. *Natural computing* 13, 4 (2014), 517–534.

[11] Hsueh-Ping Chen and Ho-Lin Chen. 2019. Self-stabilizing leader election. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*. 53–59.

[12] Edsger W Dijkstra. 1974. Self-stabilizing Systems in Spite of Distributed Control. *Communications* (1974).

[13] David Doty and David Soloveichik. 2018. Stable leader election in population protocols requires linear time. *Distributed Computing* 31, 4 (2018), 257–271.

[14] Michael Fischer and Hong Jiang. 2006. Self-stabilizing leader election in networks of finite-state anonymous agents. In *International Conference On Principles Of Distributed Systems*. Springer, 395–409.

[15] Leszek Gąsieniec, Grzegorz Stachowiak, and Przemyslaw Uznanski. 2019. Almost logarithmic-time space optimal leader election in population protocols. In *The 31st ACM Symposium on Parallelism in Algorithms and Architectures*. 93–102.

[16] Leszek Gąsieniec and Grzegorz Staehowiak. 2018. Fast space optimal leader election in population protocols. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM, 2653–2667.

[17] Gene Itkis and Leonid Levin. 1994. Fast and lean self-stabilizing asynchronous protocols. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*. 226–239.

[18] Richard M Karp and Raymond E Miller. 1969. Parallel program schemata. *Journal of Computer and system Sciences* 3, 2 (1969), 147–195.

[19] Carl Adam Petri. 1966. Communication with automata. (1966).

[20] Yuichi Sudo, Fukuhito Ooshita, Hirotsugu Kakugawa, Toshimitsu Masuzawa, Ajoy K Datta, and Lawrence L Larmore. 2018. Loosely-Stabilizing Leader Election for Arbitrary Graphs in Population Protocol Model. *IEEE Transactions on Parallel and Distributed Systems* 30, 6 (2018), 1359–1373.